言語モデルの仕組みと実装

スクリプト言語演習

本日の到達点

- 手元での Transformer の実行
- 次の 1 トークンを選ぶ処理の実装
- 簡単な応答プログラムの作成

  API キー不要。モデルをダウンロードして自分の環境で実行

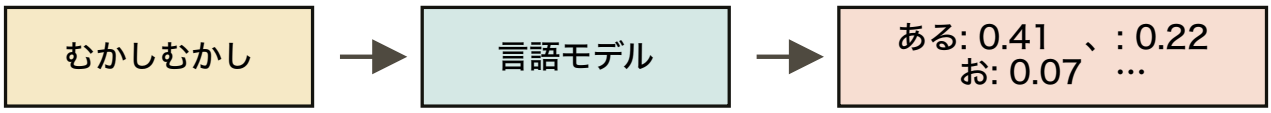
第 2 回との共通点

- 次の語の予測と、選んでつなげるループ
- 回数の表を学習済みの関数へ置き換えた点
- 第 2 回と同じ温度の利用

	第 2 回：マルコフ連鎖	今日：言語モデル
次の語の予測	`table[word]` を引く	`model(文脈)` を呼ぶ
予測の中身	数えた回数の表	学習した重み
見ている文脈	直前 1〜2 語	直前数千トークン
選び方	`random.choices` + 温度	サンプリング + 温度

言語モデルの処理

- 入力となる「ここまでの文字列」
- 出力となる、語彙ごとの確率
- 1 ステップの繰り返しによる長い文章の生成

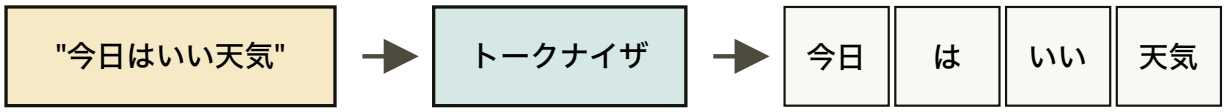


一度に 1 トークンしか返さないことによる、生成ループ

トークンと logits

- 単語とも 1 文字とも限らないトークン
- よく現れる文字列をまとめた統計的な単位
- logits に `softmax` を通して得る確率

```
probs = torch.softmax(logits, dim=-1)    # すべて 0 以上、合計 1
```



---

1 ステップの実行

- トークナイザによる文字列から ID への変換
- `model(ids)` の形：バッチ、系列長、語彙数
- 最後の位置から取得する `[0, -1]`

```
from transformers import AutoTokenizer, AutoModelForCausalLM
tok = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-0.5B-Instruct")
model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen2.5-0.5B-Instruct")

ids = tok("むかしむかし、あるところに", return_tensors="pt").input_ids
with torch.no_grad():
    logits = model(ids).logits[0, -1]

probs = torch.softmax(logits, dim=-1)
top = torch.topk(probs, 5)
# お 0.312    一 0.145    おじいさん 0.098
```

Colab ですぐに開始可能。GPU では高速、CPU でも動作

---

greedy サンプリング

- 毎回選ぶ、確率が最大のトークン
- 終端トークンでの停止
- 第 2 回の `pick_next` を `argmax` に置き換えた形

```
def generate_greedy(prompt, max_new_tokens=40):
    ids = tok(prompt, return_tensors="pt").input_ids
    for _ in range(max_new_tokens):
```

```
        with torch.no_grad():
            logits = model(ids).logits[0, -1]
        next_id = torch.argmax(logits)
        if next_id.item() == tok.eos_token_id:
            break
        ids = torch.cat([ids, next_id.view(1, 1)], dim=1)
    return tok.decode(ids[0])
```

---

greedy の弱点

- 同じ入力に対する同じ出力
- 最上位の語だけを選ぶことによる単調さ
- 分布からのランダムな選択

```
>>> generate_greedy("好きな食べ物は")
好きな食べ物は何ですか？好きな食べ物は何ですか？好きな食べ物は...
```

低い確率の語を除くための、分布の調整

温度と top-k

- 温度で割った値への softmax
- 低温で広がる差と、高温で縮まる差
- 上位 k 個への絞り込みによる低確率語の除外

```
def sample_next(logits, temperature=0.8, k=50):
    probs = torch.softmax(logits / temperature, dim=-1)
    top = torch.topk(probs, k)
    idx = torch.multinomial(top.values, 1)
    return top.indices[idx]
```

回数の `c ** (1/T)` と、確率の対数である logits の割り算の対応

発展：top-p サンプリング

- 確率の高い順の加算と、合計が p に達した時点での停止
- 偏った分布で少なく、分散した分布で多い候補
- 定番の `p = 0.9` と、`k=1` の greedy と同じ挙動

```
probs = torch.softmax(logits / temperature, dim=-1)
sorted_probs, sorted_idx = torch.sort(probs, descending=True)
cumsum = torch.cumsum(sorted_probs, dim=-1)
mask = cumsum - sorted_probs < p
```

```
next_id = sorted_idx[mask][torch.multinomial(sorted_probs[mask], 1)]
```

発展課題での実装。本編では温度と top-k で十分

素のモデルの役割

- 追加学習で得る「指示 → 応答」の書式
- 書式の文字列を生成する `apply_chat_template`
- その後の通常の生成ループ

```
messages = [
    {"role": "system", "content": " あなたは簡潔に答えるアシスタントです。"},
    {"role": "user",   "content": " 日本の首都は？"},
]
prompt = tok.apply_chat_template(
    messages, tokenize=False, add_generation_prompt=True)
```

入力: 日本の首都は
出力: 日本の首都は東京である。人口は約 1400 万人で...

会話形式の文字列

- 特殊トークンによる役割ごとの区切り
- やりとり自体を記憶しているわけではないモデル
- 「文脈を覚えている」ように見える理由

先ほどの生成ループに渡す文字列

注意点

- 長い会話による速度低下と入力上限の超過
- 0.5B モデルで頻発する、もっともらしい誤り
- 実務で使う `model.generate(...)`

問題	対処
会話が長い / 入力上限	古いやりとりを削る
同じ語の繰り返し	温度を上げる / 既出語の logits を下げる
事実ではない答え	事実確認に使わない
1 トークンずつ再計算	本来は KV キャッシュで使い回す

内部で同じ処理を効率よく行う実装

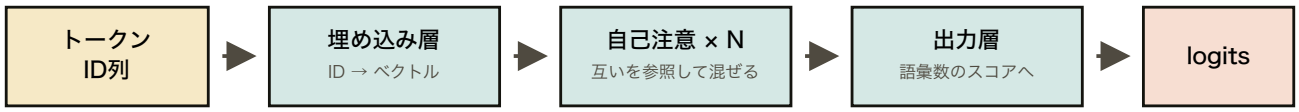
要点

- 次の 1 トークンの確率分布を返す言語モデル
- 予測・選択・連結を繰り返す生成ループ
- 履歴を毎回すべて渡して成立する会話

第 2 回との違いは、モデルの大きさと実装の速さ

Transformer の処理

- トークンのベクトル化と、注意機構による文脈の混合
- 最後に出力する語彙サイズの logits
- 理論の詳細は機械学習の講義



入出力だけの扱いで記述できる生成ループ

課題一覧

- 最初に確認する候補と確率
- greedy の後に追加する温度と top-k
- 最後に生成する、チャットテンプレートによる会話

#	課題
1	次のトークンの上位 10 候補と確率を表示する
2	`generate_greedy` を完成させ、同じ入力で結果が一致することを確かめる
3	温度付き top-k を実装し、温度 0.3 / 0.8 / 1.5 で比べる
4	チャットテンプレートで 3 往復の会話を生成する

発展：top-p サンプリング / 繰り返しペナルティ

課題の進め方

- 「Colab で開く」から課題を開き、「ドライブにコピーを保存」

- 最初の 2 セルの実行と、各課題の TODO
- 確認セルの保存と、提出前の `score()` の実行